

UNIVERSIDAD NACIONAL DE CÓRDOBA

Facultad de Ciencias Exactas, Físicas y Naturales



Trabajo Práctico Final

Red de Petri / Sistema de Agencia de Viajes

Programación Concurrente

Profesores:

VENTRE, LUIS ORLANDO

LUDEMANN, MAURICIO

Grupo: *Deadlock Dodgers*

PALACIO, LUIS ENRIQUE (40.609.862)

SAILLEN, SIMON (42.978.650)

TRACHTA, AGUSTÍN (43.271.890)

VARGAS, RODRIGO SEBASTIÁN (42.016.802)

ZUÑIGA, IVAN (43.552.893)

2024

Índice

1. DESARROLLO	2
1.1. PROPIEDADES DE LA RED	2
1.1.1. Análisis Teórico	2
1.1.2. Análisis por Software	3
2. IMPLEMENTACIÓN	4
2.1. Tabla de Estados del Sistema	4
2.2. Tabla de Eventos del Sistema	5
2.3. Responsabilidad de hilos	5
2.3.1. Algoritmo para la determinación de Hilos Máximos Activos Simultáneos .	5
2.3.2. Análisis de la Responsabilidad de los Hilos	7
3. MONITOR DE CONCURRENCIA	9
3.1. Estructura del Monitor	9
3.2. Mecanismo de Señalización: Signal and Exit	10
3.3. Diagrama de Secuencia del Monitor	10
3.4. Diagrama de Clases	10
3.5. Flujo de Disparo de una Transición	11
3.6. Independencia de la Lógica	11
4. TIEMPO	11
4.1. Análisis Teórico	12
4.2. Análisis Práctico	12
5. POLÍTICAS	13
5.1. Modelo de Resolución de Conflictos	14
5.1.1. Rol de la Política en el Monitor	14
5.1.2. Momento de Aplicación	14
5.1.3. Naturaleza Directa de la Decisión	15
5.1.4. Independencia respecto a la Señalización	15
5.2. Análisis de Resultados	15
5.2.1. Comportamiento del Conflicto en Agentes (T_2 vs T_3)	15
5.2.2. Comportamiento del Conflicto en Reservas (T_6 vs T_7)	15
5.2.3. Impacto de la Política en el Sistema	16
5.2.4. Análisis de Equidad y Priorización	16
5.2.5. Logs Registrados	17
5.2.6. Conclusión del Análisis	18
6. ANÁLISIS DE T-INVARIANTES CON REGEX	18
6.1. Resultados Regex	19
7. CONCLUSIONES	19

1. DESARROLLO

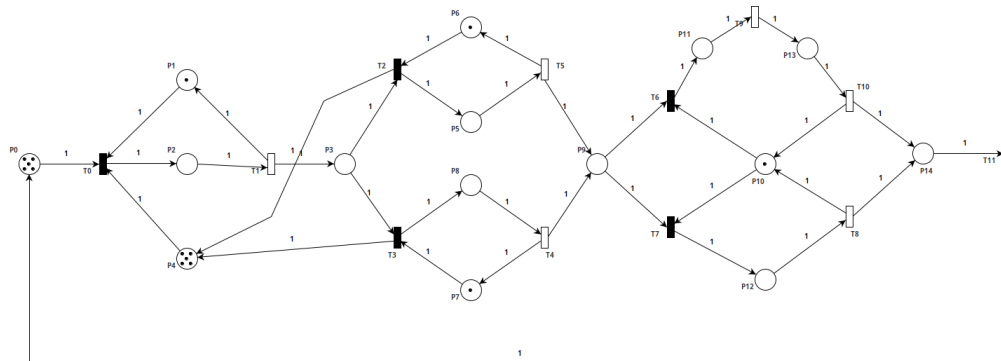


Figura 1: Red de Petri - Sistema de Agencia de Viajes

Referencias:

P1, P4, P6, P7, P10	Recursos compartidos del sistema
P0	Buffer de entrada de clientes
P2	Ingreso a la agencia
P3	Sala de espera
P5, P8	Gestión de reservas
P9	Buffer aceptación/rechazo de reserva
P12	Cancelación de reserva
P11	Confirmación de reserva
P13	Pago de reserva
P14	Instancia previa al retiro del cliente

1.1. PROPIEDADES DE LA RED

1.1.1. Análisis Teórico

- **Seguridad (Safe / Safeness):** La RdP que tenemos **no es segura**, si bien la mayoría de las plazas cumplen con la condición de tener solo 0 o 1 tokens en un determinado tiempo, tenemos unas plazas que tienen o pueden llegar a tener más de 1 token en un determinado tiempo, incumpliendo con la condición de seguridad. Para ser segura, P0 y P4 deberían tener 1 solo token, pero esto haría que la RdP solo pueda manejar a 1 solo cliente a la vez y no podrá avanzar hasta terminar con dicho cliente.
- **Vivacidad (Liveness):** La RdP que tenemos **es viva**, cada recurso compartido asegura la habilitación de las transiciones de manera cíclica.
- **Deadlock:** Nuestra RdP **no tiene deadlocks**. El diseño con retroalimentaciones y el ciclo P14 → P0 aseguran que siempre haya una secuencia de disparos posible desde cualquier marcado alcanzable, evitando estados donde todas las transiciones estén deshabilitadas.

1.1.2. Análisis por Software

Invariantes de Plaza (P-invariantes)

Petri net state space analysis results

Bounded	true
Safe	false
Deadlock	false

P-Invariants

P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14
0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	1	1	1	0
1	0	1	1	0	1	0	0	1	1	0	1	1	1	1

The net is covered by positive P-Invariants, therefore it is bounded.

Ecuaciones de Invariantes de Plaza

P-Invariant equations

$$M(P1) + M(P2) = 1$$

$$M(P2) + M(P3) + M(P4) = 5$$

$$M(P5) + M(P6) = 1$$

$$M(P7) + M(P8) = 1$$

$$M(P10) + M(P11) + M(P12) + M(P13) = 1$$

$$M(P0) + M(P2) + M(P3) + M(P5) + M(P8) + M(P9) + M(P11) + M(P12) + M(P13) + M(P14) = 5$$

Los P-invariantes, como $M(P0) + M(P2) + \dots + M(P14) = 5$ y $M(P1) + M(P2) = 1$, indican la conservación de clientes y recursos a lo largo del proceso, sugiriendo un sistema acotado y vivo. Esto refleja una agencia que maneja eficientemente 5 clientes, distribuyéndolos entre etapas y reciclando recursos, evitando deadlocks y permitiendo una operación continua.

Invariantes de Transición (T-invariantes)

T-Invariants

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11
1	1	0	1	1	0	0	1	1	0	0	1
1	1	0	1	1	0	1	0	0	1	1	1
1	1	1	0	0	1	0	1	1	0	0	1
1	1	1	0	0	1	1	0	0	1	1	1

The net is covered by positive T-Invariants, therefore it might be bounded and live.

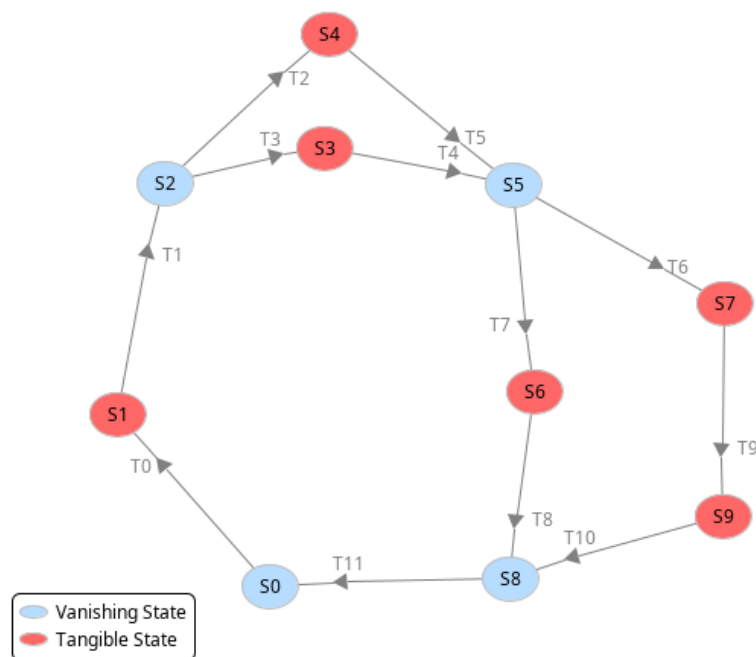
Los invariantes de transición representan secuencias cíclicas de transiciones cuya ejecución neta mantiene el marcado, reflejando procesos repetibles. Por ejemplo, el T-invariante 1 describe el flujo de un cliente desde la entrada (T0, T1) a través de la gestión alternativa (T3, T4), decisión de reserva (T7), cancelación (T8) y reingreso (T11), mientras que el T-invariante 4 cubre entrada (T0, T1), gestión inicial (T2, T5), confirmación (T6), pago (T9), salida (T10), y reingreso (T11). Los T-invariantes 2 y 3 son caminos alternativos de los anteriormente descritos.

Estos ciclos reflejan la capacidad de la agencia para manejar continuamente los 5 clientes iniciales, apoyados por retroalimentaciones, garantizando un flujo sin deadlocks.

2. IMPLEMENTACIÓN

2.1. Tabla de Estados del Sistema

Utilizando la herramienta PIPE, reduciendo el marcado inicial (para obtener un diagrama manejable), conseguimos el siguiente diagrama de estados:



Luego podemos extraer la tabla correspondiente:

Cuadro 1: Tabla de Estados Simbólicos

Estado	Plazas Involucradas	Recurso / Límite	Descripción
S_0	$P_0, (P_4 \text{ limitante})$	5 disparos	Los clientes están en el buffer inicial
S_1	$P_2, (P_1 \text{ limitante})$	1 disparo	Un cliente entró a la agencia
S_2	P_3	5 disparos	Los clientes están en cola de espera #1
S_3	$P_8, (P_7 \text{ limitante})$	1 disparo	Un cliente está siendo atendido por agente #1
S_4	$P_5, (P_6 \text{ limitante})$	1 disparo	Un cliente es atendido por agente #2
S_5	P_9	5 disparos	Los clientes están en cola de espera #2
S_6	$P_{12}, (P_{10} \text{ limitante})$	1 disparo	Un cliente cancela su reserva
S_7	$P_{11}, (P_{10} \text{ limitante})$	1 disparo	Un cliente confirma su reserva
S_8	P_{13}	1 disparo	Un cliente paga su reserva
S_9	P_{14}	5 disparos*	Los clientes se retiran de la agencia

Algunas Aclaraciones:

Los estados presentados en esta tabla son simbólicos ya que, en realidad, los mismos dependen del marcado de la RdP, pero dicha tabla resulta ser muy extensa e incómoda para extraer valor analítico.

2.2. Tabla de Eventos del Sistema

Cuadro 2: Tabla de Eventos del Sistema

Evento	Transición	Descripción
E_0	T_0	Cliente entra a la agencia, es atendido, consume P_1 y P_4 .
E_1	T_1	Cliente pasa a la sala de espera, regenera P_1 .
E_2	T_2	Cliente inicia gestión de reserva, consume P_6 y regenera P_4 .
E_3	T_3	Cliente inicia gestión de reserva alternativa, consume P_7 y regenera P_4 .
E_4	T_4	Cliente avanza en gestión alternativa, regenera P_7 .
E_5	T_5	Cliente avanza en gestión, regenera P_6 .
E_6	T_6	Cliente confirma reserva, usa P_{10} .
E_7	T_7	Cliente cancela reserva, usa P_{10} .
E_8	T_8	Cliente completa cancelación, regenera P_{10} .
E_9	T_9	Cliente realiza pago.
E_{10}	T_{10}	Cliente sale tras pago, regenera P_{10} .
E_{11}	T_{11}	Cliente regresa al buffer de entrada.

NOTA: En las descripciones sólo se hace mención al cambio de marcado de los recursos compartidos.

2.3. Responsabilidad de hilos

2.3.1. Algoritmo para la determinación de Hilos Máximos Activos Simultáneos

El objetivo del algoritmo es determinar la cantidad máxima de hilos de ejecución que pueden estar activos de forma simultánea en el sistema modelado por la RdP. Esta cantidad se obtiene

del estado de la red donde la suma de los tokens en las Plazas de Acción (PA) es máxima.

Paso 1: Obtener los Invariantes de Transición (IT) de la RdP

Los Invariantes de Transición (IT) representan las secuencias mínimas de transiciones que, al dispararse, devuelven la red a su marcado inicial. Estos ITs se asocian a las posibles trayectorias de ejecución de los procesos en el sistema.

Los ITs identificados para esta RdP son:

$$\begin{aligned} IT_1 &= \{T_0, T_1, T_3, T_4, T_7, T_8, T_{11}\} \\ IT_2 &= \{T_0, T_1, T_3, T_4, T_6, T_9, T_{10}, T_{11}\} \\ IT_3 &= \{T_0, T_1, T_2, T_5, T_7, T_8, T_{11}\} \\ IT_4 &= \{T_0, T_1, T_2, T_5, T_6, T_9, T_{10}, T_{11}\} \end{aligned}$$

Paso 2: Obtener el conjunto de plazas asociadas (PL_i) al IT que estamos analizando

El conjunto de PL_i está compuesto por todas las plazas que son pre o post condición de alguna de las transiciones pertenecientes al IT.

De esta forma, obtenemos que el conjunto de plazas asociadas son:

$$\begin{aligned} PL_1 &= \{P_0, P_1, P_2, P_3, P_4, P_5, P_6, P_7, P_8, P_9, P_{10}, P_{12}, P_{13}, P_{14}\} \\ PL_2 &= \{P_0, P_1, P_2, P_3, P_4, P_5, P_6, P_7, P_8, P_9, P_{10}, P_{11}, P_{12}, P_{13}, P_{14}\} \\ PL_3 &= \{P_0, P_1, P_2, P_3, P_4, P_5, P_6, P_7, P_8, P_9, P_{10}, P_{12}, P_{13}, P_{14}\} \\ PL_4 &= \{P_0, P_1, P_2, P_3, P_4, P_5, P_6, P_7, P_8, P_9, P_{10}, P_{11}, P_{12}, P_{13}, P_{14}\} \end{aligned}$$

Paso 3: Determinar las Plazas relacionadas a Acciones (PA) de cada IT

El conjunto de PA_i se define eliminando de PL_i todas aquellas plazas que representan elementos de control, sincronización, recursos o restricciones.

De esta forma obtenemos lo siguiente:

$$\begin{aligned} PI_{\text{recursos}} &= \{P_1, P_6, P_7, P_{10}\} \\ PI_{\text{restricciones}} &= \{P_4\} \\ PI_{\text{idle}} &= \{P_0\} \end{aligned}$$

El conjunto de plazas que no representan una acción de un hilo, entonces, es el siguiente:

$$PI_{\text{no-acción}} = \{P_0, P_1, P_4, P_6, P_7, P_{10}\}$$

Así obtenemos el conjunto final de Plazas de Acción (PA), que corresponde a las columnas de la matriz de marcados utilizadas para el cálculo, el cual es el siguiente:

$$PA = \{P_2, P_3, P_5, P_8, P_9, P_{11}, P_{12}, P_{13}, P_{14}\}$$

Paso 4: Obtener la MA

El conjunto MA es el conjunto de todos los marcados posibles de las plazas PA, obtenidos del árbol de alcanzabilidad de la RdP. Para obtener el conjunto completo de marcados posibles, empleamos la herramienta PIPE. Luego, generamos la submatriz MA al utilizar únicamente las columnas vinculadas a las PA.

Como resultado obtuvimos la siguiente matriz:

→ [MA: Conjunto de Marcados Posibles de las Plazas PA](#)

Paso 5: Buscar el Mercado Máximo (Cantidad Máxima de Hilos)

De cada mercado posible del conjunto MA, se realiza la suma de las marcas. De todas estas sumas, se debe buscar la de mayor valor. Esta será la cantidad máxima de hilos activos simultáneos en el sistema.

Implementación del script

El cálculo automatizado de la suma de marcas y la búsqueda del valor máximo se encuentra disponible en:

[Google Colab: Análisis de Concurrencia - Mercado Máximo](#)

Como resultado final, obtuvimos que la máxima cantidad de hilos activos simultáneos requeridos por el sistema es **5**.

2.3.2. Análisis de la Responsabilidad de los Hilos

La asignación de responsabilidades de los hilos es una decisión de ingeniería que traduce el modelo formal de la Red de Petri a una arquitectura de software ejecutable. Para este diseño, se ha optado por una configuración de 7 hilos, fundamentada en las reglas de segmentación estructural del paper.

Política de Diseño Adoptada

De acuerdo con la metodología, las responsabilidades de ejecución se dividen en segmentos de ejecución, que son subconjuntos de plazas de acción definidos por la presencia de conflictos (forks) y uniones (joins).

Hilo	Transiciones	Regla Aplicada
Thread-1	{0, 1}	Segmento antes del conflicto (Pre-fork en P_3).
Thread-2	{2}	Segmento tras conflicto en P_3 (Rama 1).
Thread-3	{3}	Segmento tras conflicto en P_3 (Rama 2).
Thread-4	{5}	Continuacion Rama 1.
Thread-5	{4}	Continuacion Rama 2.
Thread-6	{6, 9, 10, 11}	Segmento tras conflicto en P_9 (Rama 3).
Thread-7	{7, 8, 11}	Segmento tras conflicto en P_9 (Rama 4).

Justificación Técnica según el Algoritmo de Responsabilidad

La creación de 7 hilos responde a la necesidad de externalizar la toma de decisiones hacia el Monitor, tal como dicta el algoritmo propuesto:

1. Responsabilidad Segmentada por Conflictos (Forks)

El paper establece que ante un conflicto estructural (fork), la responsabilidad debe segmentarse:

- **Aplicación en P_3 :** La plaza de acción P_3 presenta un fork (T_2 y T_3). Siguiendo la regla, el Thread-1 asume el segmento “antes del conflicto” ($\{0, 1\}$), mientras que el Thread-2 ($\{2\}$) y el Thread-3 ($\{3\}$) asumen los segmentos “posteriores”. Seguidos por 2 hilos de continuación, Thread-4 ($\{5\}$) siguiendo a Thread-2 y Thread-5 ($\{4\}$) siguiendo a Thread-3.
- **Independencia de la Política:** Esta división asegura que la lógica del hilo sea determinística. El hilo no contiene sentencias de decisión, simplemente intenta disparar su transición. Si hay conflicto, el Monitor utiliza la Política para resolverlo.

2. Gestión del Conflicto en P_9 y Exclusión Mutua

En la plaza P_9 ocurre otro fork hacia T_6 y T_7 . Siguiendo el algoritmo, se asignan hilos por cada trayectoria (Thread-6 y Thread-7).

- **Justificación del Recurso Compartido:** Aunque el algoritmo sugiere segmentar, el Thread-6 y el Thread-7 mantienen la responsabilidad de sus caminos críticos debido a la presencia del mutex en la plaza P_{10} .
- **Control del Monitor:** El paper indica que el Monitor organiza el acceso exclusivo. Dado que T_6 y T_7 compiten por el recurso P_{10} , el Monitor siempre va a actuar como la entidad que controla la concurrencia. Los hilos 6 y 7 son independientes de la política: solicitan el disparo y es el Monitor quien, gestionando el recurso compartido, decide cuál avanza, manteniendo la independencia lógica del hilo.

3. Tratamiento de la Unión (Join) en P_{14}

El algoritmo indica que tras un join, se divide en segmentos antes de la unión y sólo un segmento extra después.

- **Optimización del Diseño:** En nuestra configuración, el Thread-6 y el Thread-7 completan la ejecución hasta T_{11} . Esto se justifica porque el mutex en P_{10} garantiza que nunca habrá tokens simultáneos en P_{14} . Al no haber concurrencia real que explotar tras la unión, no se requiere un octavo hilo “extra”, simplificando el diseño sin violar la seguridad del modelo.

Relación entre Hilos Asignados (7) y Marcado Máximo (5)

Es fundamental distinguir entre segmentos de responsabilidad y concurrencia simultánea:

- El análisis del Marcado Máximo (MA) determinó que el sistema admite un máximo de 5 hilos activos simultáneamente.
- Nuestra arquitectura asigna 7 hilos para cubrir todos los segmentos de responsabilidad definidos por los forks.

Esta configuración demuestra que, si bien la capacidad de concurrencia máxima es de 5 hilos simultáneos, la asignación de 7 hilos de responsabilidad es indispensable para lograr un desacoplamiento total. Al segmentar la ejecución de esta manera, se garantiza que la lógica de la política recaiga íntegramente en el Monitor, convirtiendo a los hilos en agentes determinísticos cuya prioridad y orden de ejecución son arbitrados exclusivamente por el modelo formal.

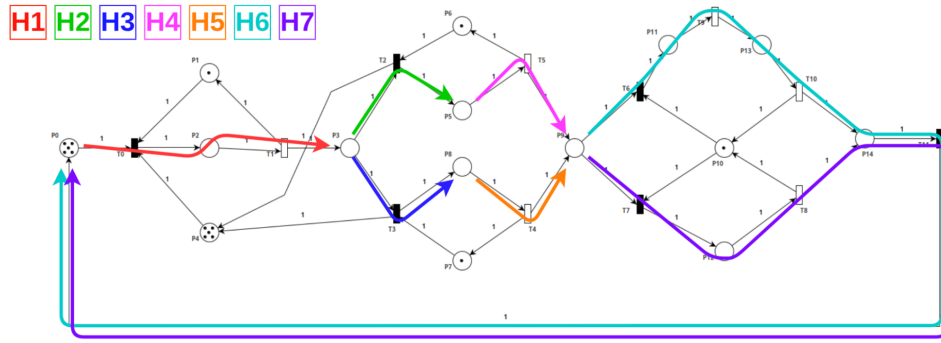


Figura 2: Representación gráfica de la asignación de hilos sobre la Red de Petri.

3. MONITOR DE CONCURRENCIA

El monitor de concurrencia es el núcleo del sistema, responsable de coordinar el acceso de los hilos a la Red de Petri y garantizar que se respeten las reglas de exclusión mutua y sincronización. La implementación se basa en una política de señalización **Signal and Exit (SX)**.

3.1. Estructura del Monitor

El monitor centraliza tres responsabilidades fundamentales:

1. **Exclusión Mutua:** Asegurar que solo un hilo a la vez opere sobre el marcado de la RdP.
2. **Gestión de Colas:** Administrar hilos bloqueados que esperan a que una transición se habilite.
3. **Semántica Temporal:** Validar que los disparos ocurran dentro de los rangos de tiempo permitidos.

Para cumplir con esto, la clase **Monitor** utiliza las siguientes estructuras internas:

- **entry:** Un semáforo binario con prioridad que actúa como el mutex de entrada al monitor.
- **queues:** Una estructura de colas que mantiene un semáforo (inicializado en 0) por cada transición de la red y una matriz de contabilidad de hilos en espera.
- **rdp:** Representación del modelo formal de la Red de Petri, que mantiene el marcado actual, las matrices de incidencia y calcula la sensibilización estructural de las transiciones.

- **policy:** El componente encargado de decidir qué hilo despertar en caso de haber conflicto.
- **time:** Gestor de la semántica temporal.

3.2. Mecanismo de Señalización: Signal and Exit

La implementación del monitor sigue la semántica de señalización conocida como **Signal and Exit**. En este modelo, cuando un hilo (el señalizador) ejecuta un disparo que sensibiliza una transición y despierta a otro hilo (el señalizado) que estaba bloqueado, el señalizador transfiere inmediatamente la posesión lógica del mutex al hilo despertado mediante un *handoff* sin retener el control del monitor.

Este enfoque se implementa a través de la clase interna **Ownership**, que modela la posesión lógica del semáforo **entry**. El método `handoff()` establece `owned = false` sin invocar `entry.release()`, manteniendo el permiso del semáforo intacto. El hilo despertado, al reanudar su ejecución desde la cola de espera de la transición, invoca `wakeFromQueue()` que establece `owned = true`, asumiendo así la posesión del mismo permiso sin necesidad de competir por él.

Este enfoque tiene dos implicancias críticas en el diseño de nuestro sistema:

- **Transferencia Directa del Mutex:** El hilo despertado no necesita re-adquirir el semáforo **entry**. La posesión se transfiere atómicamente a través del mecanismo de *handoff*, eliminando la competencia por el mutex con otros hilos que intenten ingresar al monitor.
- **Re-verificación de Condiciones:** El hilo despertado vuelve a verificar la sensibilización de la transición. Esto se debe a que entre el momento en que el hilo fue despertado y el momento en que efectúa el disparo, el estado de la red de petri puede haber cambiado.

El monitor utiliza la **Policy** para decidir a quién despertar de forma selectiva. En lugar de un broadcast, se identifica qué transiciones están sensibilizadas y tienen hilos esperando, delegando en la política la elección de la transición ganadora. La selección se realiza en el método `selectWaiterOrRelease`, que centraliza la lógica de salida del monitor: si existen hilos elegibles, efectúa el *handoff*; de lo contrario, libera el mutex directamente mediante `ownership.release()`.

3.3. Diagrama de Secuencia del Monitor

A continuación se presenta el flujo detallado de interacción entre el Worker, el Monitor y sus componentes auxiliares durante un ciclo de disparo.

[\[Clic aquí para abrir el Diagrama de Secuencia\]](#)

3.4. Diagrama de Clases

El diagrama de clases de este proyecto se encuentra a continuación.

[\[Clic aquí para abrir el Diagrama de Clases\]](#)

3.5. Flujo de Disparo de una Transición

El método principal es `fireTransition(int transition)`. Cuando un hilo intenta disparar una transición, sigue el siguiente flujo lógico:

1. **Toma del Mutex:** El hilo intenta adquirir el semáforo `entry`. Si está ocupado, espera en la cola de entrada del monitor. En iteraciones posteriores, si el hilo fue despertado mediante `handoff`, ya posee el mutex lógicamente y no necesita re-adquirirlo.
2. **Verificación de Sensibilización Estructural:** Una vez dentro, se consulta a la RdP si la transición está sensibilizada.
3. **Bloqueo (Wait):** Si la transición no está sensibilizada, el hilo incrementa el contador de espera para esa transición, libera el mutex de entrada (`release`) y se bloquea en el semáforo de su cola específica (`acquire`). Al ser despertado por un `handoff`, recupera la posesión lógica del mutex mediante `wakeFromQueue()` sin re-adquirir `entry`, y reintenta el ciclo desde el paso 1 con posesión garantizada.
4. **Evaluación Temporal:** Si está sensibilizada, se consulta al objeto `TimeRestrictions`.
 - **TOO_EARLY:** Si es demasiado pronto para disparar (no se alcanzó el α), el hilo calcula el tiempo restante, libera el mutex y duerme el tiempo restante (`Thread.sleep`). Al despertar, debe re-adquirir el mutex desde cero (no hay `handoff`).
 - **ALLOWED:** Si está dentro de la ventana, se procede al disparo.
5. **Disparo (Fire):** Se ejecuta el disparo en la RdP, actualizando el marcado. Se notifican los cambios al gestor de tiempo para resetear los *timestamps* de las transiciones afectadas.
6. **Señalización y Salida (Signal and Exit):** Antes de salir, el monitor ejecuta el método `selectWaiterOrRelease()`. Este busca todas las transiciones que ahora están sensibilizadas y tienen hilos esperando. Si hay elegibles, la `Policy` selecciona una y se produce el `handoff`: se libera el semáforo de la cola correspondiente (`signalOneWaiter`) y se transfiere la posesión lógica del mutex al hilo despertado (`handoff`), sin invocar `entry.release()`. Si no hay hilos elegibles, simplemente se libera el mutex mediante `release()`. En ambos casos, el flujo del monitor queda resuelto dentro de este método.

3.6. Independencia de la Lógica

Una característica de este diseño es que los hilos son agentes “ciegos”. Ningún hilo sabe qué está haciendo el otro, ni necesitan conocer la política de prioridad. Simplemente solicitan disparar su transición y el Monitor, junto con la Política, garantiza el progreso del sistema de forma coherente y segura.

4. TIEMPO

Teniendo el sistema funcionando, procedemos a implementar la semántica temporal; esto es, asignar tiempo a las transiciones temporales $\{T_1, T_4, T_5, T_8, T_9, T_{10}\}$.

Desarrollamos una semántica de tiempo débil (*weak-time*). Esta semántica establece que una transición temporizada puede ser disparada solo si el tiempo transcurrido desde que se habilitó

se encuentra en el rango temporal $[\alpha, \beta]$. Luego, en la implementación, se optó por tomar $\beta = \infty$, definiendo el intervalo operativo como $[\alpha, \infty)$.

La decisión de omitir un límite superior finito se tomó tras observar experimentalmente que, debido al no-determinismo del *scheduler* de la Java VM, los hilos sufrían demoras variables por cambios de contexto y competencia por el mutex del monitor. Al utilizar un β acotado, el tiempo transcurrido frecuentemente superaba dicho límite antes de que el hilo lograra ejecutar el disparo, provocando que las transiciones perdieran su habilitación temporal y los hilos quedaran suspendidos de manera crónica. Al dejar el extremo abierto con ∞ , nos aseguramos de que α funcione simplemente como el tiempo mínimo que requiere cada tarea. Así, el sistema se vuelve mucho más flexible y capaz de absorber las demoras lógicas sin que la red se trabe o deje de funcionar.

4.1. Análisis Teórico

Definición de Tiempos Asignados

Transición	Tiempo [ms]	Descripción
T_1	100	Tiempo de espera en la cola de clientes
T_4	60	Atención de agente de reservas #1
T_5	60	Atención de agente de reservas #2
T_8	80	Gestión de cancelación de reservas
T_9	40	Gestión de confirmación de reservas
T_{10}	40	Gestión de pago de reservas

Debido a la estructura de la Red de Petri, que presenta bifurcaciones para la atención y la resolución de la reserva, el Tiempo de Ciclo teórico (T_C) se calcula siguiendo los caminos lógicos independientes que un cliente puede atravesar. Dados los tiempos asignados, ambos caminos resultan equivalentes en tiempo, dando como resultado el tiempo de ciclo teórico para cualquier cliente individual:

$$T_C = \sum_i t_{min}(T_i) = 100 + 60 + 80 = 240 \quad [\text{ms}]$$

Este valor, bajo esta configuración temporal, también corresponde al tiempo por invariante.

A partir de esto, queda evidente que el cuello de botella del circuito es la transición T_1 , debido a que es la transición que individualmente requiere más tiempo para ser disparada. Al funcionar el sistema como una estructura segmentada e independiente, el sistema no podrá ser más rápido que la frecuencia de disparo de T_1 , limitando el *throughput* máximo a un cliente cada 100 ms en estado estacionario.

4.2. Análisis Práctico

El tiempo total teórico para procesar N ejecuciones de invariantes de transición se calculó de la siguiente manera:

$$T_{\text{teorico}} = T_C + (N - 1) \times t_{\text{bottleneck}}$$

Donde:

- T_C : Es el tiempo de ciclo para cualquier cliente individual.
- N : La cantidad de invariantes de tiempo.
- $t_{\text{bottleneck}}$: El tiempo de la transición mas lenta.

Para la realización de la tabla se tomaron 5 muestras de ejecuciones de cada escenario y se calculo el promedio entre ellas.

#	Escenario	Tiempo Teorico [ms]	Tiempo Practico [ms]	Desvio (%)
1	Tiempo balanceado	18.740	18.497	1,29
2	Etapas final 4 veces mas lenta	37.360	36.934	1,14
3	Agentes 4 veces mas lentos	22.620	22.466	0,68
4	Cola de clientes 2 veces mas rapida	14.990	14.549	2,94

Análisis de Resultados:

- **Escenario 1 (Base):** Funciona como punto de referencia. El tiempo total de ejecución está delimitado por la transición T_1 (100 ms), la cual actúa como el cuello de botella primario del *pipeline*, dosificando el flujo de tokens hacia el resto de las etapas independientes.
- **Escenario 2:** Al incrementar T_9 y T_{10} a 160 ms cada una, la sub-rama de confirmación pasa a requerir 320 ms en serie. Como un porcentaje de los clientes se bifurca obligatoriamente por este camino, la etapa de cierre se convierte en el nuevo cuello de botella absoluto del sistema. Esto genera una acumulación de hilos en las variables de condición previas, duplicando el tiempo total de ejecución de la red.
- **Escenario 3:** Al ralentizar simultáneamente ambos canales de atención en paralelo ($T_4 = T_5 = 240$ ms), la capacidad de procesamiento combinada de esa etapa se reduce a la mitad de su frecuencia individual, procesando idealmente un token cada 120 ms. Al ser este tiempo mayor al de T_1 , la etapa de agentes desplaza a T_1 como cuello de botella.
- **Escenario 4:** Al reducir el tiempo de T_1 a la mitad (50 ms), el cuello de botella original se acelera. Sin embargo, el cuello de botella del *pipeline* se desplaza inmediatamente a la siguiente etapa más lenta disponible en la red (en este caso, la gestión de cancelación $T_8 = 80$ ms o la suma de confirmación/pago), alcanzando el límite de saturación natural de la arquitectura concurrente actual.

5. POLÍTICAS

En la red de Petri utilizada para modelar el sistema de agencia de viajes existen conflictos estructurales entre determinadas transiciones. Un conflicto estructural ocurre cuando dos o más transiciones comparten al menos un lugar de entrada, por lo que el disparo de una de ellas consume los tokens necesarios e impide el disparo simultáneo de las demás.

En el modelo implementado se identificaron dos grupos principales de conflicto:

- **Conflicto entre agentes de reservas:**
 - T_2 : representa la atención por el agente superior

- T_3 : representa la atención por el agente inferior
- **Conflicto entre decisiones de reservas:**
 - T_6 : representa la confirmación de una reserva
 - T_7 : representa la cancelación de una reserva

Ambos pares de transiciones comparten al menos una plaza de entrada, lo que da lugar a los conflictos estructurales mencionados.

El proyecto requiere la implementación de tres tipos de políticas para resolver dichos conflictos:

- Política balanceada
- Política priorizada
- Sin política*

Estas políticas fueron implementadas mediante un módulo independiente denominado `policy`, el cual es consultado por el monitor al momento de seleccionar qué transición despertar entre aquellas que se encuentran habilitadas.

5.1. Modelo de Resolución de Conflictos

5.1.1. Rol de la Política en el Monitor

En la implementación desarrollada, la política no interviene directamente en la decisión de disparo de una transición, sino en el mecanismo de señalización del monitor.

Específicamente, la política se aplica cuando el monitor debe decidir qué hilo despertar entre aquellos que se encuentran en espera y cuyas transiciones están estructuralmente habilitadas.

De esta manera, la política actúa como un mecanismo de planificación (*scheduling*) que condiciona directamente la evolución del sistema.

5.1.2. Momento de Aplicación

El flujo de ejecución del sistema es el siguiente:

1. Un hilo dispara una transición dentro del monitor.
2. Se actualiza el marcado de la Red de Petri.
3. Se identifican las transiciones que:
 - Están sensibilizadas.
 - Tienen hilos en espera.
4. Si existen múltiples alternativas habilitadas simultáneamente, se consulta a la política.
5. La política selecciona una transición.
6. Se despierta un hilo asociado a dicha transición y se le transfiere el mutex mediante handoff.

Es importante destacar que, gracias al mecanismo de handoff, la decisión de la política tiene efecto directo: el hilo despertado recibe la posesión lógica del mutex y es el próximo en ejecutar sin interferencia de otros hilos.

5.1.3. Naturaleza Directa de la Decisión

En Signal and Exit la política tiene control directo sobre qué hilo se ejecuta a continuación. Esto se debe a que:

- El hilo despertado recibe el mutex inmediatamente mediante handoff, sin pasar por la cola de entrada.
- No existen otros hilos compitiendo por el acceso en el momento del despertar.
- La selección se realiza sobre transiciones, y la política determina cuál de las transiciones sensibilizadas con hilos esperando será atendida.

Por lo tanto, la política implementa un control directo y efectivo sobre la evolución del sistema.

5.1.4. Independencia respecto a la Señalización

La política actúa únicamente en la selección de la transición a despertar, manteniendo una separación clara entre Sincronización (monitor) y Decisión (política). Esta separación permite modificar el comportamiento del sistema sin afectar su correctitud concurrente.

5.2. Análisis de Resultados

Se realizaron múltiples ejecuciones del sistema completando 186 invariantes en cada corrida, con el objetivo de evaluar el comportamiento de las políticas implementadas frente a los conflictos estructurales definidos.

5.2.1. Comportamiento del Conflicto en Agentes (T_2 vs T_3)

En el caso del conflicto entre agentes de reservas (T_2 y T_3), se observa que todos los disparos ocurren bajo condiciones de conflicto (aunque no se trate necesariamente de conflicto efectivo), esto ocurre por la necesidad de decidir de la política, se debe contemplar al conflicto como siempre efectivo para mantener consistencia con los resultados.

5.2.2. Comportamiento del Conflicto en Reservas (T_6 vs T_7)

El conflicto entre decisiones de reservas (T_6 vs T_7) también presenta que todos los disparos ocurren bajo condición de conflicto, esto es así debido a la estructura de la red, solo se puede tomar 1 camino posible y se utiliza el mismo recurso compartido.

5.2.3. Impacto de la Política en el Sistema

La política actúa sobre dos aspectos del sistema:

- **Selección de wake-up:** Determina qué hilo despierta de entre los que esperan por transiciones sensibilizadas.
- **Deferral de reservas:** Para T_6 y T_7 , la política puede impedir que un hilo dispare aunque su transición esté sensibilizada, forzándolo a ceder el monitor hasta que la transición preferida sea seleccionada.

La influencia de la política es directa cuando ambas transiciones en conflicto tienen hilos esperando simultáneamente. En ese caso, la selección es determinista, la política elige cuál despertar y cuál deferir.

Sin embargo, la política no puede actuar cuando solo una de las transiciones tiene hilos esperando, ya que no hay alternativa posible. En estos casos, el sistema se rige por la estructura de la red de petri y el orden de llegada de los hilos.

5.2.4. Análisis de Equidad y Priorización

Política balanceada (BALANCED)

Se espera distribución equitativa ($\approx 50\%/50\%$) para ambos pares de conflicto.

Política priorizada (PRIORITIZED)

Se espera:

- T_2 aproximadamente 75 % del total de disparos de agentes (relación 3:1).
- T_6 aproximadamente 80 % del total de disparos de reservas (relación 4:1).

Sin Política (NONE)

Agregamos esta ausencia de política para mayor análisis, la siguiente decide sobre que transición sera disparada en caso de haber conflicto de manera aleatoria. Por la forma de la red la distribución de elección termina con valores similares a la política balanceada.

5.2.5. Logs Registrados

===== RESULTADOS =====

Policy mode: BALANCED

--- AGENTES (T2 vs T3) ---

Total disparos: 186

En conflicto: 186

T2 (superior): 93

T3 (inferior): 93

% Superior (conflicto): 50.00%

Sin conflicto: 0

TOTAL GLOBAL:

T2: 93

T3: 93

% Superior (global): 50.00%

--- RESERVAS (T6 vs T7) ---

Total disparos: 186

En conflicto: 186

T6 (confirmadas): 93

T7 (canceladas): 93

% Confirmadas (conflicto): 50.00%

Sin conflicto: 0

TOTAL GLOBAL:

T6: 93

T7: 93

% Confirmadas (global): 50.00%

=====

===== RESULTADOS =====

Policy mode: PRIORITIZED

--- AGENTES (T2 vs T3) ---

Total disparos: 186

En conflicto: 186

T2 (superior): 140

T3 (inferior): 46

% Superior (conflicto): 75.27%

Sin conflicto: 0

TOTAL GLOBAL:

T2: 140

T3: 46

% Superior (global): 75.27%

--- RESERVAS (T6 vs T7) ---

Total disparos: 186

En conflicto: 186

T6 (confirmadas): 149

T7 (canceladas): 37

% Confirmadas (conflicto): 80.11%

Sin conflicto: 0

TOTAL GLOBAL:

T6: 149

T7: 37

% Confirmadas (global): 80.11%

=====

===== RESULTADOS =====

Policy mode: NONE

--- AGENTES (T2 vs T3) ---

Total disparos: 186

En conflicto: 0

T2 (superior): 0

T3 (inferior): 0

Sin conflicto: 186

TOTAL GLOBAL:

T2: 97

T3: 89

% Superior (global): 52.15%

--- RESERVAS (T6 vs T7) ---

Total disparos: 186

En conflicto: 0

T6 (confirmadas): 0

T7 (canceladas): 0

Sin conflicto: 186

TOTAL GLOBAL:

T6: 93

T7: 93

% Confirmadas (global): 50.00%

=====

5.2.6. Conclusión del Análisis

El análisis experimental permite concluir que:

- La política implementada influye de manera efectiva en la evolución del sistema, tanto mediante la selección de wake-up como mediante el mecanismo de deferral para reservas.
- La distribución observada se aproxima a los valores teóricos (50 %, 75 %, 80 %).

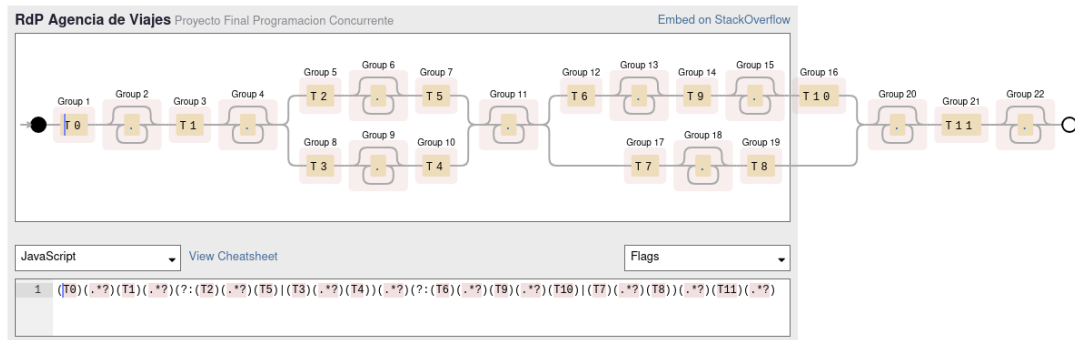
Esto valida tanto el diseño del monitor como la correcta implementación de las políticas requeridas.

6. ANÁLISIS DE T-INVARIANTES CON REGEX

Se utilizó la siguiente expresión regular para detectar las invariantes de transición (concatenadas como $T_0T_1T_2\dots$):

```
1 (T0)(.*) (T1)(.*) (?: (T2)(.*) (T5) | (T3)(.*) (T4)) (.*?) (?: (T6)(.*) (T9) (.*?) (T10) | (T7) (.*?) (T8)) (.*?) (T11)(.?)
```

El cual se ve expresado gráficamente en [Debuggex](#):



El patrón modela el orden lógico del invariante con ambas bifurcaciones luego de la sala de espera y de la aceptación/rechazo de reserva, respectivamente; el cual corresponde a las cuatro invariantes de transición mencionadas anteriormente.

Aplicamos $(.*)$, que representa cualquier secuencia intermedia de transiciones, que luego se guardan eliminando las transiciones que ya forman parte de un invariante. El uso de ‘?’ permite que el cuantificador sea no codicioso (lazy), consumiendo lo mínimo necesario para hacer match con un invariante.

Los paréntesis representan grupos ‘(...)’, con ello capturamos partes del match para reutilizar luego, en este caso lo usamos para consumir iterativamente el log, es decir, una vez encontrado un match de transiciones, se cuenta y se eliminan estas para continuar analizando hasta vaciarlo. Esto último se realiza mediante el reemplazo, expresado en Python de la siguiente manera:

```
1 replacement = r"\g<2>\g<4>\g<6>\g<9>\g<11>\g<13>\g<15>\g<18>\g<20>\g<22>"
```

Los índices 2, 4, 6... representan solo a los interleavings. Con ‘re.subn(..., count = 1)’ se reemplaza una ocurrencia por iteración, eliminando las transiciones que componen el invariante y

preservando los interleavings restantes. El conteo se obtiene repitiendo este proceso hasta que no se encuentren más coincidencias.

Las bifurcaciones del modelo se representan con alternativas 'or' (A|B) agrupadas en '(?:...)' para que se aplique sobre el tramo completo sin generar grupos capturantes extra, esto mantiene estable la numeración de grupos necesaria para el reemplazo.

6.1. Resultados Regex

Resultado de la ejecución del script de Python utilizando como entrada los logs:

```
Archivo de log cargado: logs/run.log
Cantidad de invariantes cargados: 4
Primeras transiciones: T0T1T2T5T6T0T9T1T2T5T10T0T11T1T3T4T6T0T9T1T2T5T10T

=== Reporte de invariantes ===
Invariantes detectados: 186
Invariantes esperados: 186
Resultado: OK

Detalle por tipo de invariante:
T0-T1-T2-T5 / T6-T9-T10-T11: 112
T0-T1-T2-T5 / T7-T8-T11: 28
T0-T1-T3-T4 / T6-T9-T10-T11: 37
T0-T1-T3-T4 / T7-T8-T11: 9
```

Figura 4: Salida de ejecución de REGEX sobre logs.

El analizador encuentra 186 invariantes completos, pero si quedan transiciones remanentes se verán final del log.

7. CONCLUSIONES

A partir del desarrollo realizado se logró implementar una simulación concurrente de la red de Petri correspondiente al sistema de agencia de viajes, utilizando un monitor como mecanismo central de sincronización. La implementación permitió coordinar el disparo de transiciones, resolver conflictos mediante políticas y controlar el acceso concurrente a los recursos compartidos representados por las plazas de la red.

El uso de hilos diferenciados según los caminos e invariantes de transición permitió aprovechar el paralelismo del modelo, manteniendo al mismo tiempo una ejecución controlada por el monitor. Cada hilo tuvo una responsabilidad asociada a una secuencia de transiciones, mientras que el monitor se encargó de verificar la sensibilización, aplicar las restricciones temporales y decidir qué transición podía dispararse en cada momento. Esto permitió separar la lógica concurrente de la lógica propia de la red de Petri.

Además, se incorporó semántica temporal en las transiciones correspondientes, lo que permitió representar de manera más realista los tiempos asociados a ciertas operaciones del sistema. A

partir de las ejecuciones realizadas, se pudo observar cómo los tiempos definidos influyen directamente en el comportamiento global de la simulación, en el orden de disparo de las transiciones y en la distribución de los invariantes obtenidos.

La implementación de políticas permitió analizar distintos criterios de resolución de conflictos. En particular, se pudo comparar el comportamiento del sistema bajo criterios balanceados y priorizados, verificando mediante los resultados finales la cantidad de clientes atendidos por cada rama del modelo y la proporción de reservas confirmadas o canceladas.

Por último, el uso de logs permitió validar posteriormente la ejecución mediante expresiones regulares. El análisis confirmó la presencia de los 186 invariantes de transición esperados.

En conclusión, el trabajo permitió aplicar los conceptos principales de programación concurrente sobre un modelo formal basado en Redes de Petri, integrando sincronización con monitores, manejo de hilos, políticas de decisión, restricciones temporales y verificación posterior mediante logs. Los resultados obtenidos muestran que la simulación respeta los invariantes definidos y que el sistema implementado logra representar correctamente el comportamiento concurrente del modelo propuesto.